

SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery Solutions

Student Name: Keval Prajapati

TASK 1: Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.

REQUEST CONFIGURATION

GET https://jsonplaceholder.typicode.com/posts/1

b) Record the status code you get back.

RESPONSE STATUS

200 OK Time: 142 ms | Size: 1.25 KB

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto"
}
```

c) In one sentence, explain why this request works even with nothing in the Headers tab.

Answer: This request works perfectly because the GET method is intended exclusively for fetching resource data from the server, meaning it does not send a payload body and therefore requires no special structural metadata or Content-Type headers to guide the server.

TASK 2: Filtering With Query Parameters

- a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.
- b) Confirm the URL automatically updates to include ?postId=1.

REQUEST CONFIGURATION

GET`https://jsonplaceholder.typicode.com/comments?postId=1`

- c) Now combine two filters at once on /todos: userId = 1 and completed = false.

COMBINED REQUEST URL

GET`https://jsonplaceholder.typicode.com/todos?userId=1&completed=false`

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  }
]
```

- d) In one sentence, explain what changed in the results compared to not using any params.

Answer: Implementing the query parameters successfully narrows down the server's output, rendering a targeted dataset that perfectly isolates matching criteria instead of returning the entire unfiltered collections.

TASK 3: Sending Data With Headers & a Body

- a) Send a POST request to /posts.
- b) In the Headers tab, add: Content-Type: application/json
- c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

REQUEST CONFIGURATION & JSON PAYLOAD (NEW EXAMPLE DATA)

POST https://jsonplaceholder.typicode.com/posts

Headers: Content-Type: application/json

```
{
  "title": "Advanced Postman Frameworks",
  "body": "Developing reliable and automated API integration test suites using
environments and global scripts.",
  "userId": 42
}
```

- d) Record the status code and the new id the server assigned.

RESPONSE OUTPUT

201 Created

```
{
  "title": "Advanced Postman Frameworks",
  "body": "Developing reliable and automated API integration test suites using
environments and global scripts.",
  "userId": 42,
  "id": 101
}
```

TASK 4: Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.

REQUEST CONFIGURATION

POST `https://jsonplaceholder.typicode.com/posts`

Headers: None

b) Compare the response to Task 3 — is it different? Record what you observe.

Feature Matrix	Task 3 (With Header)	Task 4 (Without Header)
Content-Type Header	application/json	Omitted / None
HTTP Status Code	201 Created	201 Created
Payload Processing	Correctly parsed by server	Omitted from parsing (Empty Object reflection)
Response ID	101 (With fields returned)	101 (Only ID generated)

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

Answer: Headers serve as crucial metadata instructions that allow the backend framework to recognize how to correctly unpack incoming byte streams. While standard practice sandboxes like JSONPlaceholder inherently adapt to raw body payloads gracefully, real production environments rely completely on these explicitly declared contents. Skipping the Content-Type header on live commercial systems will trigger formatting failures or prompt an absolute rejection from strict application servers.

TASK 5: PUT vs. PATCH

a) Send a PUT request to `/posts/1` that replaces the entire post (include id, title, body, and userId in the body).

PUT REQUEST & PAYLOAD (NEW EXAMPLE DATA)

PUT `https://jsonplaceholder.typicode.com/posts/1`

```
{
  "id": 1,
  "title": "Overhauled Documentation Blueprint",
  "body": "This entire structural architecture has been successfully overwritten and updated utilizing a full PUT request.",
  "userId": 88
}
```

PUT RESPONSE STATUS: 200 OK

```
{
  "id": 1,
  "title": "Overhauled Documentation Blueprint",
  "body": "This entire structural architecture has been successfully overwritten and updated utilizing a full PUT request.",
  "userId": 88
}
```

b) Send a PATCH request to the same `/posts/1` that changes only the title.

PATCH REQUEST & PAYLOAD

PATCH `https://jsonplaceholder.typicode.com/posts/1`

```
{
  "title": "Partially Modified Title Edition"
}
```

PATCH RESPONSE STATUS: 200 OK

```
{
  "userId": 1,
  "id": 1,
  "title": "Partially Modified Title Edition",
  "body": "quia et suscipit  
suscepit recusandae consequuntur expedita et cum  
reprehenderit molestiae ut ut quas totam
```

```
nostrum rerum est autem sunt rem eveniet architecto"
}
```

c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

Answer: A PUT request supplies a comprehensive, fully complete resource model to replace the existing entry entirely, whereas a PATCH request sends exclusively the distinct field modifications intended for isolated update.

TASK 6: The Mini Feedback System

Scenario: Testing a backend feedback flow using comments.

Step	HTTP Request Target	Header Flags Included	Status Code Received
a	GET /comments?postId=1	None / Inherited	200 OK
b	POST /comments	None (Omitted entirely)	201 Created
c	POST /comments	Content-Type: application/json	201 Created
d	POST /comments	Content-Type + Authorization: Bearer demo-token	201 Created

f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

Answer: On a secured commercial deployment, the infrastructure would instantly intercept the request to decode and validate the crypto signature of the provided Bearer token against an active directory session. Had a fake or malformed token like 'demo-token' been transmitted, the production gateway would immediately terminate the transmission, answering with an explicit 401 Unauthorized or 403 Forbidden status code to block unverified write operations.